STUDENT MANAGEMENT SYSTEM

Python Internship — Week 6 Final Project Report

Student Name	Arijit Maiti (edit if needed)
Internship Domain	Python Development
Week	6 of 6
Project Type	Menu-driven console application
Technologies	Python 3, JSON, Git, GitHub

Prepared for internship submission. Replace or add your own screenshots and GitHub repository link before submitting.

1. Introduction

The Student Management System is a beginner-friendly Python application designed to store and manage student records through a simple text menu. It demonstrates practical use of the concepts covered during a six-week Python internship, including data types, control flow, functions, data structures, file handling, exception handling, and object-oriented programming.

2. Problem Statement

Keeping student information in handwritten notes or scattered files can make it difficult to find, update, and maintain records. This project provides a simple digital approach to add, view, search, update, and delete student details, with records saved between sessions.

3. Project Objectives

• Build a working Python application with a clear menu.

• Store student ID, name, age, course, and city.

• Allow users to add, view, search, update, and delete records.

• Validate user input and handle common errors safely.

• Persist records in a JSON file.

• Organize the project for GitHub and document setup and usage.

4. Features

• Add a student after checking for a blank or duplicate ID.

• Display all student records.

• Search by student ID.

• Update selected fields while keeping unchanged fields intact.

• Delete records only after user confirmation.

• Load and save records using JSON.

• Handle invalid menu choices, invalid ages, malformed JSON, and file errors.

5. Technologies Used

• Python 3 — application logic and object-oriented design.

• json — serializes student records into JSON format.

• pathlib — manages the data-file path.

• Git — local version control.

• GitHub — remote repository hosting and project sharing.

6. Python Concepts Implemented

Concept	Implementation
Variables and data types	Student ID, name, age, course, city, strings, integers, lists, dictionaries.
Input/output	input() and print() provide menu interaction.

Concept	Implementation
Conditionals and loops	if/elif/else validates menu choices; while loop keeps the menu running.
Functions and methods	Separate methods handle add, view, search, update, delete, load, and save.
Data structures	A list holds Student objects; dictionaries represent records for JSON.
File handling	with open() reads and writes students.json.
Exception handling	try/except handles invalid input, JSON parsing, and file errors.
OOP	Person and Student classes, constructors, inheritance, private ID, and methods.
Git/GitHub	Repository initialization, commits, and pushing the project online.

7. Project Workflow

• Start the program by running main.py.

• The application loads existing records from students.json.

• The user chooses an action from the menu.

• The selected method validates input and updates the in-memory student list.

• After a successful change, the program saves the list to students.json.

• The menu repeats until the user selects Exit.

8. Installation and Execution

• Install Python 3 on the computer.

• Open Command Prompt or VS Code terminal in the project folder.

• Run: py main.py (or python main.py).

• Choose menu options 1–6 and follow the prompts.

• The students.json file stores records. Keep a backup before manually editing it.

9. Git and GitHub Workflow

Create a new empty repository on GitHub named Student-Management-System. Open a terminal in the project folder and run the commands below. Replace the sample repository URL with your own URL.

```
git init
git status
git add .
git commit -m "Initial Student Management System project"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/Student-Management-System.git
git push -u origin main
```

If Git does not know the author identity, configure it with your own details using git config --global user.name and git config --global user.email. Never put passwords or access tokens in source files.

10. Screenshots / Output

Insert genuine screenshots from your own computer before submission. Suggested screenshots:

• Screenshot 1: program menu after running py main.py.

• Screenshot 2: successful Add Student operation.

• Screenshot 3: View All Students showing a sample record.

• Screenshot 4: students.json containing saved data.

• Screenshot 5: GitHub repository page showing uploaded files.

Screenshot placeholders are not included as if they were real evidence. Capture your own output and place the images in the screenshots folder.

11. Challenges Faced

• Preventing duplicate student IDs.

• Handling invalid age input without crashing the application.

• Saving data so it remains available after the program closes.

• Organizing the source code and documentation for GitHub.

12. Future Improvements

• Create a graphical user interface using Tkinter.

• Use SQLite for more structured data storage.

• Add login and role-based access control.

• Export student lists to CSV or PDF.

• Add sorting, filtering, and course-wise reports.

13. Conclusion

This project demonstrates how fundamental Python programming skills can be combined to create a useful student-record application. It uses functions, collections, validation, exception handling, JSON persistence, and OOP. Publishing the code on GitHub also introduces version control and professional project documentation.

14. Repository Link

GitHub Repository: [Paste your actual GitHub repository URL here]

15. Student Presentation / Viva Preparation

What problem does the project solve?

It provides a simple way to maintain student records digitally.

Where is data stored?

In the students.json file in the project folder.

Why use JSON?

It is readable and can store lists of dictionary-like records without external packages.

Which OOP concepts are used?

Classes, objects, constructors, inheritance, encapsulation, and methods.

How are errors handled?

try/except blocks handle invalid input and file/JSON errors.

What can be improved?

A GUI, database, login system, and export/report features can be added.